XGBoost in Practice: Investigating Tree Depth and Learning Rate Using a Heart Disease Classification Dataset

Introduction

Machine learning regulations are increasingly used to support predictive analysis in different sectors and application areas. For instance, healthcare, finance, social sciences, etc are now using machine learning algorithms for different purposes. In modern machine learning methods, the gradient boosting algorithms have gained quite a lot of popularity for their performance and flexibility. Extreme gradient boosting or XGBoost is one of the most popular implementations of gradient boosting.

XGBoost is a gradient boosting implementation that is more efficient, flexible, and portable than existing implementations and performs better than other algorithms. XGBoost is widely used in data science competitions and real-world machine learning applications because of its strong predictive power and efficient optimisation.

Nonetheless, XGBoost's performance is dependent on hyperparameter selection. The depth of the tree and learning rate are two important hyperparameters. The complexity of an individual tree is controlled by tree depth, while the learning rate controls how strongly each tree contributes to the final prediction.

This tutorial aims to show how these hyperparameters affect the performance of the model. Using a heart disease classification dataset from Kaggle, the tutorial looks at how different tree depths and learning rates impact predictive accuracy, training behaviour, and model stability.

Gradient Boosting and XGBoost

Boosting is an ensemble learning technique that builds models sequentially. Instead of training a single model, boosting constructs a sequence of weak learners, where each new learner focuses on correcting the errors of previous models. By gradually improving predictions, boosting algorithms can produce powerful predictive models.

Gradient Boosting is a boosting method that optimises a loss function using gradient descent. At each iteration, a new model is trained to predict the residual errors produced by the previous models. By repeatedly modelling these residuals, the algorithm progressively reduces prediction error.

XGBoost extends traditional gradient boosting with several improvements. These include regularisation techniques that reduce overfitting, parallel processing for faster computation, and optimised tree construction methods. These enhancements allow XGBoost to train models efficiently even on large datasets.

Despite these advantages, the performance of XGBoost depends strongly on appropriate hyperparameter tuning. Selecting suitable values for parameters such as tree depth and learning rate is therefore essential for achieving optimal model performance.

Dataset Description

The dataset used in this tutorial is the **Heart Disease Prediction Dataset**, which is publicly available on Kaggle. The dataset contains medical and demographic attributes used to predict whether a patient has heart disease.

Some of the key features in the dataset include:

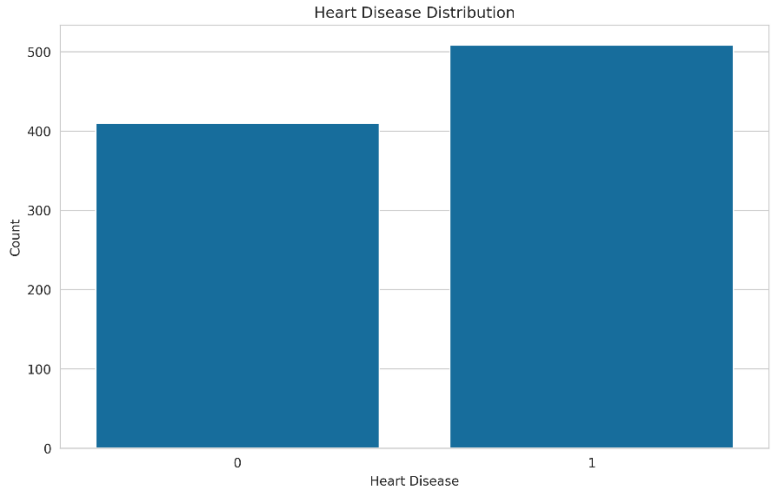
- Age of the patient
- Sex
- Chest pain type
- Resting blood pressure
- Cholesterol level
- Maximum heart rate achieved.
- Exercise-induced angina.
- ST depression related to exercise

Heart disease, whether the patient suffers from the affliction, is the target variable. The task of modelling is, therefore, a binary classification task where the model predicts whether the patient will have a heart attack based on the features provided.

The dataset was pre-processed prior to training the ML model. The label encoding was applied to categorical variables to be used for the XGBoost algorithm in the model. The data was split into two datasets, one for training and another for testing purposes.

Exploratory Data Analysis

Exploratory data analysis provides insight into the structure of the dataset and helps identify potential patterns before building predictive models.



The heart disease dataset has both positives and negatives as shown in the distribution plot. It is important to understand the distribution of the target variable since imbalanced datasets affect the model evaluation metrics.

Looking at the dataset structure in addition to the target distribution should give you an idea of whether you need to encode or normalise the data. XGBoost is a tree-based model that does not require feature scaling which is easy for preprocessing.

XGBoost Model

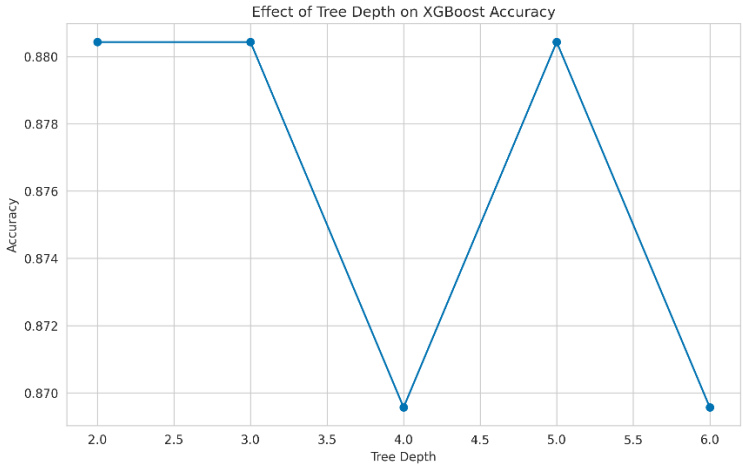
A XGBoost model using default hyperparameters was trained to evaluate performance. The model adopted a tree depth of 3 and a learning rate of 0.1.

This model provides a first estimate of predictive accuracy before conducting hyperparameter experiments. XGBoost can get strong classification score through little tuning, as shown by the results of the baseline model. A model must first be established so that the impact of any changes in hyperparameter can be evaluated.

Investigating the Effect of Tree Depth

The complexity of individual decision trees in the boosting ensemble is controlled by tree depth. If a tree is deeper, it means the variance is larger too which means that such a model will be complex more overfitting.

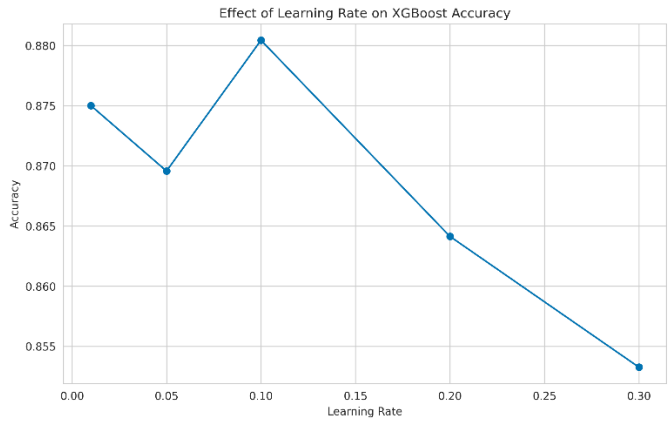
The XGBoost model was trained with depth values from two to six to analyse the effect of tree depth. The performance of every single model was identified on the test data.



The results show that increasing tree depth can improve model performance up to a certain point. However, excessively deep trees may reduce generalisation performance because the model begins to memorise patterns in the training data rather than learning general relationships.
This experiment demonstrates the importance of selecting an appropriate tree depth when training XGBoost models.

Investigating the Effect of Learning Rate

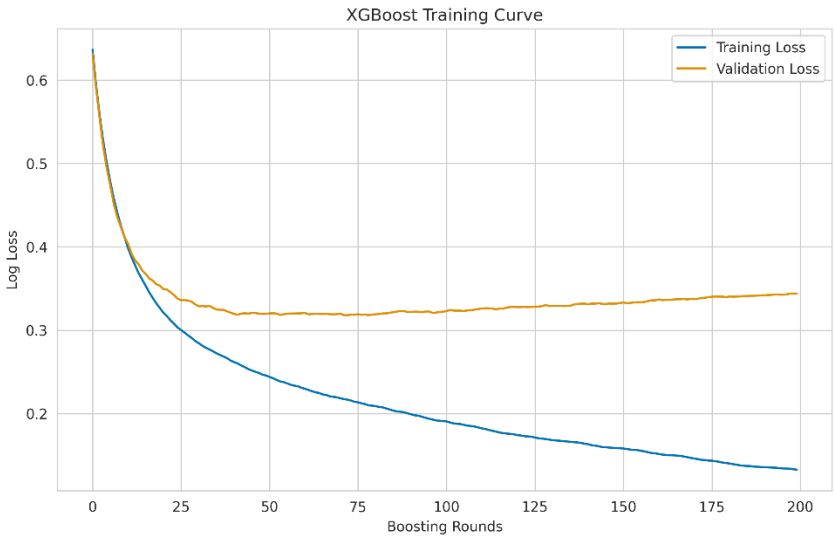
The learning rate controls how much each new tree contributes to the overall model prediction. Smaller learning rates reduce the influence of individual trees, allowing the model to learn gradually. Larger learning rates increase the contribution of each tree, which can accelerate training but may lead to unstable predictions. To evaluate the effect of learning rate, models were trained using learning rates ranging from 0.01 to 0.30.



The results demonstrate that moderate learning rates produce better model performance. Extremely small learning rates may slow down learning, while large learning rates can lead to unstable predictions.
This experiment highlights how learning rate influences the speed and stability of boosting algorithms.

Training Curve Analysis

To better understand how the model learns during training, training curves were generated to track the log-loss metric across boosting iterations.



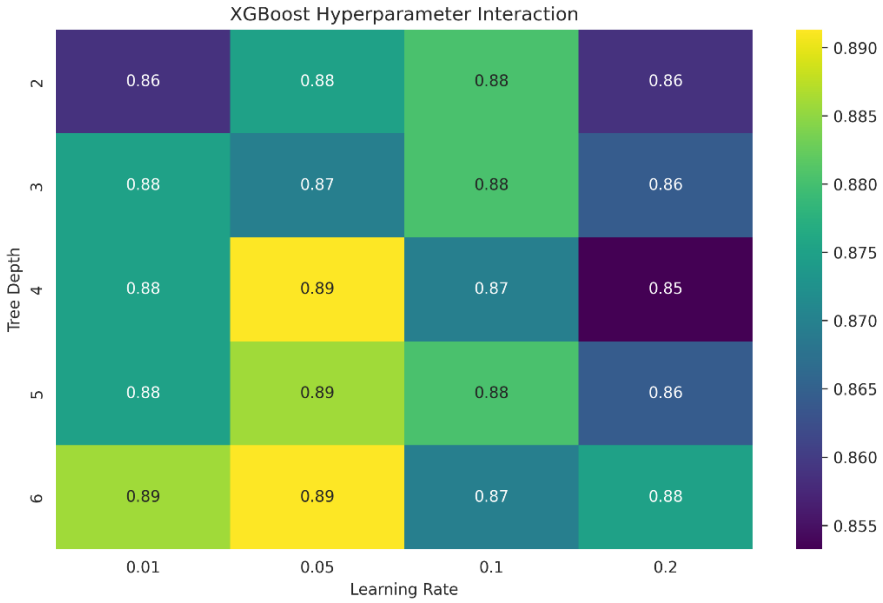
As more boosting rounds are added, model's loss is decreasing as shown in the training curve. This suggests the model's predictions improve incrementally with the addition of more trees into the model.

When the values of training loss and validation loss are not same, overfitting can be indicated. If the validation loss starts to increase while the training loss decreases, the model may be overfitting the training data.

Hyperparameter Interaction Analysis

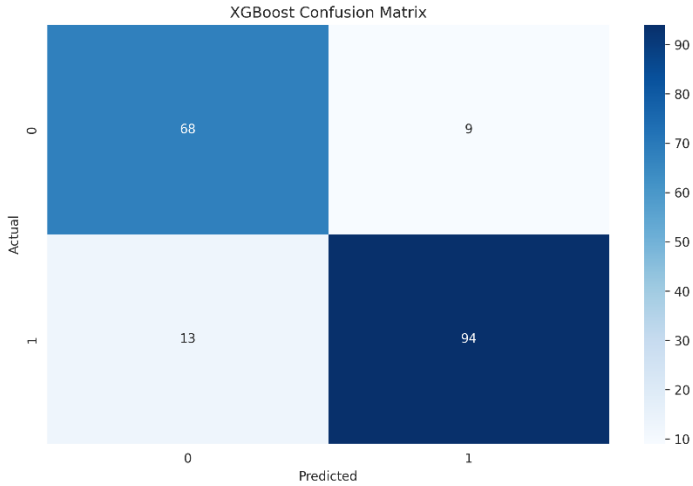
In addition to analysing individual hyperparameters, it is useful to examine how hyperparameters interact with one another. For example, the optimal tree depth may depend on the learning rate used during training.

To explore this relationship, a hyperparameter interaction experiment was conducted in which models were trained using multiple combinations of tree depth and learning rate.



The heatmap highlights combinations of hyperparameters that produce higher classification accuracy. Such visualisations are useful for identifying optimal parameter settings and understanding how different hyperparameters influence model behaviour.

Model Evaluation



The confusion matrix summarizes how well your classification model performs on a set of test data. True positives refer to correctly identified cases of patients with heart disease while true negatives refer to correctly classified healthy patients.

Other calculations performed in the research paper included additional metrics like precision, recall, and F1-score of model performance evaluation.

	Precision	Recall	F1- score	Support
0	0.84	0.88	0.86	77
1	0.91	0.88	0.90	107

Discussion

The experiments conducted in this tutorial demonstrate how key XGBoost hyperparameters influence model performance. Tree depth affects the complexity of individual trees, while learning rate controls the contribution of each boosting iteration.

The results show that moderate values of both parameters produce better model performance. Extremely high tree depths or large learning rates may lead to overfitting or unstable predictions.

The visualisations presented in this tutorial provide a clear explanation of how these parameters influence model behaviour and training dynamics.

Limitations

Even though the tests show how hyperparameters affect the performance of models, there are several issues. The experiment results depend on what data is being analysed, and so does the optimum hyperparameters selection.

Also, this tutorial examined just two hyperparameters. The model's performance may also be significantly influenced by other parameters, like the number of estimators and subsampling rate.

Future Work

Future work could extend this analysis by investigating additional hyperparameters such as subsampling rates and column sampling. Another direction would be to compare XGBoost with other gradient boosting frameworks such as LightGBM or CatBoost to evaluate their relative performance on the same dataset.

Conclusion

This tutorial explored the practical use of XGBoost for classification tasks using a heart disease dataset. The analysis investigated how tree depth and learning rate influence model performance, training behaviour, and predictive accuracy.

The experiments demonstrate that hyperparameter tuning plays a critical role in the performance of boosting algorithms. By analysing training curves and hyperparameter interactions, it is possible to better understand how the model learns and how different parameter settings influence its behaviour.

Overall, XGBoost provides a powerful and flexible framework for machine learning tasks, and careful hyperparameter tuning can significantly enhance its predictive performance.

Reference

Chen, T. and Guestrin, C. (2016) XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.

Friedman, J. (2001) Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), pp. 1189–1232.

Pedregosa, F. et al. (2011) Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

Géron, A. (2022) Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow. O'Reilly Media.

Kaggle (2020) Heart Disease Prediction Dataset. Available at:
https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction